
Fixed-point combinator, term calculus, operational semantics, Kleene recursion theorem, operator equation, automatic differentiation, deep equilibrium models, Green’s function

A TERM CALCULUS WITH FIXED POINTS AS PRIMITIVE: THE T-TRANSFORM AND THE UNIFICATION OF OPERATOR EQUATIONS

ZIXI LI

Sun Yat-sen University
e-mail address: lizx93@mail2.sysu.edu.cn

ABSTRACT. We define a term calculus extending the simply-typed lambda-calculus with three self-reference primitives—quotation, evaluation, and fixed points—in which every operator equation on a vector space is solved by a single construct: the fixed point of the T-transform, defined as $T(u) = u$ minus η times the operator applied to u . We prove that among all affine term-forming operations, the T-transform is the canonical form satisfying the fixed-point-is-solution property. We give a decomposition into differential and constraint operators that unifies scalar equations, initial value problems, boundary value problems, and time-dependent PDEs as instances of the same operator equation differing only in the choice of constraint. We show that reverse-mode automatic differentiation is expressible as a term-to-term transformation within the calculus, and that for implicit-layer models, both the forward pass and the gradient are fixed points of the T-transform. The entire system is implemented in approximately 1,500 lines of code. The solver kernel is under 30 lines.

1. INTRODUCTION

1.1. **Motivation.** Consider two programs:

- $f(x) = x + 1$ — a term that computes a value.
- `solve(x**3 - 2*x - 5 = 0)` — a call to a library that finds a value.

The first is a term in a programming language. The second is a black-box procedure, invisible to the type system, opaque to composition. This distinction is not a mathematical necessity; it is an engineering artifact.

We propose a term calculus in which solving an equation is not an operation applied to a term, but *a term itself*. The fixed-point combinator `Fix`—not a numerical algorithm, but a constructor in the syntax—constitutes the sole mechanism for solving equations. Given any operator A on a vector space and a scalar $\eta \neq 0$, the term $\text{Fix}(\lambda u. u - \eta \cdot A[u])$ is the solution to $A[u] = 0$. The proof that this term satisfies the equation is two lines and holds for any A on any space.

1.2. The T-Transform. The T-transform is a *residual operator embedding*: rather than solving $A[u] = 0$ directly, we embed the equation into a discrete dynamical system whose stationary points are exactly the zeros of A . Define, for any operator A on a vector space X and scalar $\eta \neq 0$:

$$T_{A,\eta}[u] \equiv u - \eta \cdot A[u]. \quad (1.1)$$

This induces the operator flow $u_{k+1} = T_{A,\eta}(u_k)$, whose continuous analog is $\partial_t u = -A[u]$ (explicit Euler discretization with time step η). The central observation of this paper is that the fixed point of this flow is exactly the solution:

$$\text{Fix}(T) = T(\text{Fix}(T)) \implies A[\text{Fix}(T)] = 0. \quad (1.2)$$

The forward direction of (1.2) holds by the operational semantics of Fix . The reverse direction—that $A[\text{Fix}(T)] = 0$ follows—is a two-line algebraic cancellation (Theorem 3.2).

We characterize the T-transform within the class of affine residual operators (Theorem 3.4): among all binary term-forming operations $\Phi(u, v)$ affine in both arguments, the form $\Phi(u, A[u]) = u - \eta \cdot A[u]$ is the canonical representative satisfying the fixed-point-is-solution property. Any other linear combination $\alpha u + \beta A[u]$ fails to entail $A[\text{Fix}(\Phi)] = 0$ from $\text{Fix}(\Phi) = \Phi(\text{Fix}(\Phi))$ without additional hypotheses on A .

1.3. Contributions.

- (1) **A term calculus with Fix as primitive** (Section 2). We extend the λ -calculus with Quote , Eval , and Fix as syntactic constructors whose operational semantics directly encode self-reference. The evaluation rule $\text{Fix}(f) \rightarrow f \text{ Fix}(f)$ implements Kleene’s first recursion theorem as a reduction step.
- (2) **The T-transform and its canonicity** (Section 3). We characterize the T-transform within the class of affine residual operators: among all linear combinations $\alpha u + \beta A[u]$, the fixed-point-is-solution property holds iff $\alpha = 1$ and $\beta \neq 0$. The T-transform is the canonical representative of this class; its form is forced by the requirement that $\text{Fix}(\Phi) = \Phi(\text{Fix}(\Phi))$ entail $A[\text{Fix}(\Phi)] = 0$ without additional hypotheses on A .
- (3) **Unification of equation types via (D, C)** (Section 4). We show that scalar equations, IVPs, BVPs, and time-dependent PDEs differ only in the choice of the constraint operator C in the decomposition $(D, C)u = (f, g)$. The solver never branches on equation type.
- (4) **Automatic differentiation as a term operator** (Section 5). We define a Wengert tape within the calculus and prove that reverse-mode AD is a term-to-term transformation. The gradient of any expression is computable as $\text{Fix}(T)$ applied to the adjoint equation.
- (5) **DEQ gradient as nested Fix** (Section 6). We prove that for implicit-layer models, both the forward pass $z^* = \text{Fix}(\lambda z. f(z, x, \theta))$ and the gradient $g^* = \text{Fix}(\lambda g. \partial L / \partial z^* + g \cdot \partial f / \partial z)$ are instances of $\text{Fix}(T)$.

1.4. Related Work. Kleene [Kle52] introduced the first recursion theorem as a meta-theorem about general recursive functions; the second recursion theorem appeared earlier in [Kle38]. Barendregt [Bar84] treats the Y combinator as a definable term within the untyped λ -calculus. Our approach differs: Fix is a *primitive* constructor, not definable from application and abstraction, and its operational semantics *is* Kleene’s first recursion theorem.

Plotkin [Plo77] and Scott [Sco76] gave domain-theoretic semantics for fixed-point operators in PCF and LCF. Our calculus is untyped: all terms inhabit a single syntactic domain, without a separate type hierarchy for functions and values.

The Wengert tape [Wen64] is the standard data structure for reverse-mode automatic differentiation. Its formulation as a term-to-term transformation within a λ -calculus, rather than as an external procedure, is new.

Bai *et al.* [BKK19] introduced Deep Equilibrium Models and the implicit-function-theorem gradient. The use of fixed-point iteration for the gradient computation was presented as a numerical technique. We recast both forward and backward passes as instances of the same $\text{Fix}(T)$ construct, and we prove that this recasting is not an analogy—it is a formal identity within the calculus.

Chen *et al.* [CRBD18] unified ODE solvers with residual networks. Their adjoint method propagates gradients through a continuous-time dynamical system. Our $\text{Fix}(T)$ framework subsumes their forward pass but replaces the adjoint ODE with another fixed point, eliminating the need to integrate backward in time.

2. THE TERM CALCULUS

2.1. Syntax. Let $\mathcal{V}$ be a countable set of variable names. The set $\mathcal{T}$ of terms is defined inductively:

Definition 2.1 (Terms).

$t ::= \text{Var}(x)$	variable, $x \in \mathcal{V}$
$\text{Lam}(x, t)$	abstraction, $\lambda x. t$
$\text{App}(t_1, t_2)$	application
$\text{Quote}(t)$	reification
$\text{Eval}(t_1, t_2)$	self-application
$\text{Fix}(t)$	fixed point
$\text{Nat}(n)$	natural number, $n \in \mathbb{N}$
$\text{Cons}(t_1, t_2)$	pair
$\text{Fun}(f, S)$	quantity in space S
$\text{Space}(\text{name}, D, \delta)$	vector space
$\text{Integ}(t, a)$	integral operator (Volterra)
$\text{Integ}(t, a, b)$	integral operator (Fredholm)
$\text{Diff}(t)$	derivative operator

The constructors fall into four groups:

- **Computation:** Var , Lam , App — the pure λ -calculus.
- **Self-reference:** Quote , Eval , Fix — reification, self-application, and fixed points.
- **Data and quantities:** Nat , Cons , Fun , Space — natural numbers, pairs, vector-space points, and the spaces themselves.
- **Analysis:** Integ , Diff — integral and derivative as first-class terms.

There is no primitive type distinction between “functions” and “values”. $\text{Fun}(f, S)$ is a term like any other. A scalar 3.14 is $\text{Fun}(\lambda x. 3.14, \text{SPACE_R})$ —a constant function on the real line. This unification is essential: it ensures that the T-transform $T_{A,\eta}[u] = u - \eta \cdot A[u]$ is well-typed for any operator A , regardless of whether u is a scalar (optimization), a function (PDE solving), or a parameter vector (training).

2.2. Operational Semantics. The reduction relation $\rightarrow$ is defined by the following rules. Evaluation proceeds to weak head normal form.

Definition 2.2 (Reduction rules). β -**reduction**:: $\text{App}(\text{Lam}(x, t), s) \rightarrow t[x := s]$

Self-application:: $\text{Eval}(\text{Quote}(t), s) \rightarrow t \ s$

Fixed point:: $\text{Fix}(f) \rightarrow f \ \text{Fix}(f)$

Integral application (Volterra):: $\text{App}(\text{Integ}(\text{Lam}(\tau, b), a), x) \rightarrow F(x)$
 where $F(x) = \int_a^x b[\tau := \xi] \, d\xi$

Integral application (Fredholm):: $\text{App}(\text{Integ}(\text{Lam}(\tau, b), a_0, a_1), x) \rightarrow G(x)$
 where $G(x) = \int_{a_0}^{a_1} K(x, \xi) \cdot b[\tau := \xi] \, d\xi$

Derivative application:: $\text{App}(\text{Diff}(\text{Lam}(x, b)), p) \rightarrow D(b)[x := p]$
 where $D(b) = \lim_{h \rightarrow 0} \frac{b[x:=p+h] - b[x:=p-h]}{2h}$

Fun evaluation:: $\text{App}(\text{Fun}(f, S), x) \rightarrow f(x)$

The recursion depth of Fix expansion is bounded by a global parameter MAX_FIX to ensure termination of evaluation. The formal correctness of fixed-point constructions does not depend on this bound; it is a practical measure for the interpreter.

Remark 2.3 (Semantic interpretation of Fix). The reduction rule $\text{Fix}(f) \rightarrow f \ \text{Fix}(f)$ defines the *operational* semantics of Fix . For the theorems of Sections 3–4, the *denotational* interpretation of Fix is the Banach fixed point in a complete metric space: when X is a complete metric space and $T : X \rightarrow X$ is a contraction, $\text{Fix}(T)$ denotes the unique element $u^* \in X$ satisfying $u^* = T(u^*)$ (Theorem 3.7). The operational rule and the denotational interpretation are compatible: the reduction sequence $u_{k+1} = T(u_k)$ converges to the denotational fixed point when T is a contraction. The syntactic theorem $\text{Fix}(T) = T(\text{Fix}(T))$ (Theorem 3.2) holds in both interpretations—as a reduction step in the operational semantics, and as the defining equation of the Banach fixed point in the denotational semantics. We do not mix domain-theoretic, rewrite, and metric interpretations: throughout this paper, the denotational semantics of Fix is the Banach fixed point over complete metric spaces.

2.3. The Environment and Numerical Backend. Evaluation occurs in an environment Env that maps variable names to values. The core evaluator $\text{eval_term}(t, \text{env}, fd)$ evaluates t to weak head normal form via repeated reduction. The parameter fd tracks the current Fix depth to enforce the MAX_FIX bound.

Definition 2.4 (Environment). Env is a chained lookup structure: each frame maps a finite set of variable names to terms, with a parent pointer to the enclosing frame. The lookup function $\text{lookup}(\text{env}, x)$ searches frames from innermost to outermost.

The evaluation loop handles Fix specially. When the evaluator encounters $\text{Fix}(f)$, it applies one step of expansion:

$$\text{eval_term}(\text{Fix}(f)) \rightarrow \text{eval_term}(\text{App}(f, \text{Fix}(f)), fd + 1).$$

If fd exceeds MAX_FIX , evaluation terminates with the current approximation. This is a pragmatic bound—the formal correctness of $\text{Fix}(T)$ as a solution (Theorem 3.2) holds regardless of how many expansion steps are taken, because the identity $\text{Fix}(T) = T(\text{Fix}(T))$ is an equation in the syntax, not an asymptotic limit. The MAX_FIX parameter is set to 5000 in the implementation, which is far larger than any fixed-point iteration that converges in practice.

2.4. Arithmetic and the Critical Compilation Rule. The calculus lifts arithmetic pointwise over function spaces. Binary operations $(+, -, \times)$ are defined via a single dispatcher `_binop` that handles all combinations:

Definition 2.5 (Pointwise arithmetic). For terms f, g in the same space, `_binop`(f, g, op) computes $h(x) = op(f(x), g(x))$ pointwise:

- (1) $(\text{Fun}, \text{Fun}) \rightarrow \text{Fun}$: compose the callables pointwise.
- (2) $(\text{Fun}, \text{Lam}) \rightarrow \text{Fun}$: **compile Lam to Fun first**, then combine as (Fun, Fun) .
- (3) (Lam, Lam) with the same parameter: push the operation symbolically into the body (for construction, not iteration).

The second rule is critical. A naïve implementation would push the `Fun` into the `Lam` body: `Lam(p, App(..., p))`. After n iterations, the `Lam` nesting depth is $O(n)$, and evaluation traverses n nested closures. The compile rule avoids this entirely: `_compile_lam` samples the `Lam` on a grid spanning the function’s domain in one shot, returning a flat `Fun` backed by piecewise linear interpolation, which never triggers nested `eval_term` calls. This is the single design rule that makes the fixed-point iteration practical.

After each Euler step, `flatten_term` samples the resulting `Fun` on a fresh grid and wraps it in a new interpolation, breaking the closure chain. Without flattening, the n -th iterate chains through n closures; with flattening, each iterate is a flat lookup table whose evaluation cost is $O(1)$ regardless of iteration count.

2.5. Free Variables and Substitution.

Definition 2.6 (Free variables). The set of free variables $\text{FV}(t)$ is defined by structural induction:

$$\begin{aligned}
 \text{FV}(\text{Var}(x)) &= \{x\} \\
 \text{FV}(\text{Lam}(x, t)) &= \text{FV}(t) \setminus \{x\} \\
 \text{FV}(\text{App}(t_1, t_2)) &= \text{FV}(t_1) \cup \text{FV}(t_2) \\
 \text{FV}(\text{Fix}(t)) &= \text{FV}(t) \\
 \text{FV}(\text{Quote}(t)) &= \text{FV}(t) \\
 \text{FV}(\text{Eval}(t_1, t_2)) &= \text{FV}(t_1) \cup \text{FV}(t_2) \\
 \text{FV}(\text{Integ}(t, a)) &= \text{FV}(t) \cup \text{FV}(a) \\
 \text{FV}(\text{Integ}(t, a, b)) &= \text{FV}(t) \cup \text{FV}(a) \cup \text{FV}(b) \\
 \text{FV}(\text{Diff}(t)) &= \text{FV}(t)
 \end{aligned}$$

Substitution $t[x := s]$ is capture-avoiding and defined for all term types. The only binding construct is `Lam`; `Integ(Lam( $\tau$ ,  $b$ ), ...)` binds τ in b only during integration—the binder is structural, not a separate binding form.

2.6. Fix Implements Kleene.

Theorem 2.7 (Fix implements Kleene). *The reduction rule $\text{Fix}(f) \rightarrow f \text{ Fix}(f)$ directly implements Kleene’s first recursion theorem: for any computable function f , there exists a term $\text{Fix}(f)$ such that $\text{Fix}(f)$ reduces to $f(\text{Fix}(f))$.*

Proof. The reduction rule IS the statement. The existence of the fixed point is not a meta-theorem about the calculus; it is a reduction step within the calculus. Fix is the witness whose self-application generates the Kleene fixed point by construction. $\square$

Corollary 2.8 (Self-reference completeness). *The triple (Quote, Eval, Fix) is complete for self-referential constructions: any computable function of its own description can be expressed.*

Proof. Quote reifies a term as data accessible to other terms. Eval applies a reified term to an argument, implementing the universal function. Fix constructs the self-application point. Together these three operations suffice to implement any interpreter, self-modifying program, or meta-circular evaluator [Bar84]. $\square$

2.7. Space Polymorphism. A Space term carries its own algebraic structure:

Definition 2.9 (Space).

$$\text{Space}(\text{name}, D, \delta)$$

where $D \subseteq \mathbb{R}^n$ is the domain and $\delta : \mathcal{T} \times \mathcal{T} \rightarrow \mathbb{R}^+$ is the distance function.

The distance function δ is polymorphic: for finite-dimensional spaces $\delta(a, b) = |a - b|$ (Euclidean), for function spaces $\delta(f, g) = (\int |f - g|^2)^{1/2} (L^2)$.

Lemma 2.10 (Distance polymorphism). *The solver evaluates $\text{sp.distance}(a, b)$ without branching on the space type. The metric is a property of the space, not of the solver.*

Proof. Each Space instance carries a closure implementing δ . The solver calls this closure uniformly. The dispatch to Euclidean or L^2 metric occurs inside the closure, invisible to the solver’s control flow. $\square$

The importance of space polymorphism for the T-transform framework cannot be overstated. The T-transform $T_{A, \eta}[u] = u - \eta \cdot A[u]$ involves subtraction—the operation $u - \eta \cdot A[u]$ must be meaningful for whatever space u inhabits. When solving a scalar equation, $u \in \mathbb{R}$ and subtraction is the usual one. When solving a BVP, u is a function on $[0, 1]$ and subtraction must be pointwise. When training a neural network, u is a parameter vector and $\eta \cdot A[u]$ is a gradient step in $\mathbb{R}^p$. The solver—the single function $\text{solve}(A, \eta)$ that returns $\text{Fix}(T(A, \eta))$ —calls the same arithmetic operations regardless. Space polymorphism makes this possible: the space object dispatches the arithmetic, not the solver.

3. THE T-TRANSFORM

3.1. Definition and Core Theorem. The T-transform is a *residual operator embedding*: it reinterprets the static operator equation $A[u] = 0$ as the fixed-point condition of a discrete dynamical system on the function space X . The central observation is that *solving an equation* and *finding a stationary point of an operator flow* are the same mathematical act.

Given an operator $A : X \rightarrow X$, the T-transform induces the one-parameter family of discrete flows:

$$u_{k+1} = T_{A,\eta}(u_k), \quad T_{A,\eta}[u] \equiv u - \eta \cdot A[u].$$

The continuous-time analog is the operator differential equation $\partial_t u = -A[u]$, of which $T_{A,\eta}$ is the explicit Euler discretization with time step η . Every fixed point of the flow is a zero of A , and every zero of A is a fixed point of the flow—the correspondence is exact (Theorem 3.2). What is unified across equation types is not the structure of the equation but the *evolution in function space* that solves it.

Definition 3.1 (T-transform). For any operator A on a vector space X and any scalar $\eta \neq 0$:

$$T_{A,\eta}[u] \equiv u - \eta \cdot A[u].$$

In the term calculus, $T_{A,\eta}$ is the abstraction $\text{Lam}(u, \text{sub}(u, \text{mul}(\text{R}(\eta), \text{App}(A, u))))$.

Theorem 3.2 (Fixed-point is solution). *For any operator A and any $\eta \neq 0$:*

$$\text{Fix}(T_{A,\eta}) = T_{A,\eta}(\text{Fix}(T_{A,\eta})) \implies A[\text{Fix}(T_{A,\eta})] = 0.$$

Proof.

$$\begin{aligned} \text{Fix}(T) &= T(\text{Fix}(T)) && \text{(definition of Fix)} \\ &= \text{Fix}(T) - \eta \cdot A[\text{Fix}(T)] && \text{(definition of } T) \\ \implies 0 &= -\eta \cdot A[\text{Fix}(T)] && \text{(subtract Fix}(T)) \\ \implies A[\text{Fix}(T)] &= 0 && (\eta \neq 0) \end{aligned}$$

□

The proof is two algebraic steps. It applies to *any* operator A on *any* vector space, without assumptions about linearity, continuity, or differentiability. The term $\text{Fix}(T_{A,\eta})$ is a *syntactic* theorem that $A[u] = 0$ has a solution—the existence of the solution is guaranteed by the reduction rule $\text{Fix}(f) \rightarrow f \text{ Fix}(f)$.

Remark 3.3 (Strength of the embedding). Theorem 3.2 establishes a *dynamical-system embedding*: the equation $A[u] = 0$ is equivalent to the fixed-point condition $u = T_{A,\eta}(u)$. This is not a claim of structural isomorphism across equation classes (which would require categorical or operator-theoretic equivalence proofs), but a claim of *uniform representability*: every operator equation admits the same syntactic form $\text{Fix}(T_{A,\eta})$, and the semantic content resides entirely in the choice of A . The unification is at the level of the solving process (evolution in function space), not at the level of the equation structure.

3.2. Canonicity.

Theorem 3.4 (Canonicity of the T-transform). *Among all term-forming operations $\Phi(u, v)$ affine in both arguments, the form $\Phi(u, A[u]) = u - \eta \cdot A[u]$ is the canonical representative satisfying the fixed-point-is-solution property $\text{Fix}(\Phi) = \Phi(\text{Fix}(\Phi)) \implies A[\text{Fix}(\Phi)] = 0$.*

Proof. Let $\Phi(u, v) = \alpha u + \beta v$ with α, β scalars, representing the most general linear term-forming operation on the pair $(u, A[u])$. Then:

$$\begin{aligned} \text{Fix}(\Phi) &= \Phi(\text{Fix}(\Phi)) \\ &= \alpha \cdot \text{Fix}(\Phi) + \beta \cdot A[\text{Fix}(\Phi)] \\ \Rightarrow (1 - \alpha) \cdot \text{Fix}(\Phi) &= \beta \cdot A[\text{Fix}(\Phi)] \end{aligned}$$

For the solution condition $A[\text{Fix}(\Phi)] = 0$ to follow from $\text{Fix}(\Phi) = \Phi(\text{Fix}(\Phi))$ without additional assumptions on A or $\text{Fix}(\Phi)$, we require:

- (1) $\beta \neq 0$: otherwise the equation gives no information about A .
- (2) $\alpha = 1$: otherwise $\text{Fix}(\Phi) = \frac{\beta}{1-\alpha} \cdot A[\text{Fix}(\Phi)]$ relates $\text{Fix}(\Phi)$ to $A[\text{Fix}(\Phi)]$ rather than forcing $A[\text{Fix}(\Phi)] = 0$.

With $\alpha = 1$ and $\beta \neq 0$:

$$0 \cdot \text{Fix}(\Phi) = \beta \cdot A[\text{Fix}(\Phi)] \implies A[\text{Fix}(\Phi)] = 0.$$

Setting $\beta = -\eta$ gives the T-transform. □

Corollary 3.5. *The T-transform is canonical up to the choice of η . Different choices of η correspond to different preconditioners of the same dynamical system $u_{t+1} = u_t - \eta \cdot A[u_t]$.*

Remark 3.6 (Scope of the characterization). Theorem 3.4 characterizes the T-transform within the restricted hypothesis class of *affine* term-forming operations $\Phi(u, A[u]) = \alpha u + \beta A[u]$. It does not preclude the existence of nonlinear preconditioners $\Phi(u, A[u])$ (involving, e.g., coordinate transforms $\varphi(u)$ or state augmentation with $A^2[u], A^3[u], \dots$) that also satisfy the fixed-point-is-solution property. The theorem establishes that *within the affine class*, the T-transform is the canonical and essentially unique representative; extending the characterization to broader operator classes is a direction for future work.

The significance of Theorem 3.4 is that the T-transform is not an arbitrary design choice. It is the *canonical* linear combination $u - \eta \cdot A[u]$ that makes the fixed-point-is-solution property hold without additional hypotheses on A .

3.3. Convergence.

Theorem 3.7 (Contractivity). *Let X be a complete metric space with distance δ , and let $A : X \rightarrow X$ be Lipschitz with constant L . If $|\eta| < 2/L$, then $T_{A,\eta}$ is a contraction and $\text{Fix}(T_{A,\eta})$ is the unique fixed point.*

Proof. For any $u, v \in X$:

$$\begin{aligned} \delta(T(u), T(v)) &= \delta(u - \eta A[u], v - \eta A[v]) \\ &= \|(I - \eta A)(u - v)\| \\ &\leq \|I - \eta A\| \cdot \|u - v\|. \end{aligned}$$

For $\|I - \eta A\| < 1$, we require $|\eta| < 2/L$ when A satisfies the Baillon–Haddad condition $\langle Au - Av, u - v \rangle \geq \frac{1}{L} \|Au - Av\|^2$. Under this condition T is a strict contraction. By the Banach fixed-point theorem [Ban22], $\text{Fix}(T)$ exists and is unique. □

Remark 3.8. The formal correctness of $\text{Fix}(T)$ (Theorem 3.2) does *not* depend on convergence. The fixed point *is* the solution as a syntactic theorem. Whether a particular numerical evaluation converges—that is, whether the sequence $u_{k+1} = T(u_k)$ reaches a neighborhood of $\text{Fix}(T)$ within finitely many iterations—is a separate property of A and η , governed by Theorem 3.7. This separation of syntactic correctness from numerical convergence is a key architectural feature of the calculus.

3.4. The η Parameter.

Theorem 3.9 (η unification). *The scalar η in $T[u] = u - \eta \cdot A[u]$ simultaneously plays three roles:*

- (1) **Time step** in differential equation solvers ($\eta = \Delta t$);
- (2) **Learning rate** in gradient-based optimization ($\eta = \text{lr}$);
- (3) **Damping factor** in fixed-point iteration.

These are not analogies. They are the same parameter in the same equation.

Proof. In each case, η controls the step size in the dynamics $u_{t+1} = u_t - \eta \cdot A[u_t]$.

- For $A = d/dt$ (time derivative), $\eta = \Delta t$ recovers Euler’s method.
- For $A = \nabla_{\theta} \mathcal{L}$ (gradient), η is the learning rate in gradient descent.
- For $A = I - f$ (fixed-point residual), η damps the iteration $u_{t+1} = (1 - \eta)u_t + \eta f(u_t)$.

The optimal η satisfies the same Armijo condition $\mathcal{L}(u_t - \eta A[u_t]) \leq \mathcal{L}(u_t) - c \cdot \eta \cdot \|A[u_t]\|^2$ regardless of whether $\mathcal{L}$ is a physical energy, a loss function, or a residual norm. A schedule η_t (line search, cosine annealing) improves convergence identically across all three contexts. $\square$

The unification of η is visible directly in the implementation. The training algorithm updates parameters via two lines:

$$\begin{aligned} \delta &\leftarrow \eta \cdot \nabla_{\theta} \mathcal{L} \\ \theta &\leftarrow \theta - \delta \end{aligned}$$

This is exactly the T-transform $\theta \leftarrow \theta - \eta \cdot \nabla_{\theta} \mathcal{L}$, with η playing the role of learning rate. The same η appears in the solver construction:

$$T_{A,\eta} = \lambda u. u - \eta \cdot A[u], \quad \text{solve}(A, \eta) = \text{Fix}(T_{A,\eta}).$$

Both the PDE solver and the neural network trainer invoke the same $\text{Fix}(T)$ construct. The trainer is not analogous to the solver; the trainer *is* the solver, with $A = \nabla_{\theta} \mathcal{L}$.

4. THE UNIFIED OPERATOR EQUATION

4.1. The (D, C) Decomposition. Every classical equation type decomposes into an interior law D and an exterior constraint C :

Definition 4.1 (Unified operator equation).

$$(D, C)u = (f, g)$$

where

- $D : X \rightarrow Y$ is the differential operator (interior law);
- $C : X \rightarrow Z$ is the constraint operator (boundary / initial slice);
- $f \in Y, g \in Z$ are the data.

The constraint operator C selects a slice of the unknown function. The equation type is determined entirely by the signature of C :

C takes	Equation type	Canonical example
$\emptyset$	Scalar equation	$x^3 - 2x - 5 = 0$
$u(a)$	Initial value problem	$y' = y, y(0) = 1$
$(u(a), u(b))$	Boundary value problem	$-u'' = 1, u(0) = u(1) = 0$
$(u(a), u(b)) \times \mathbb{N}_t$	Time-dependent PDE	$u_t = u_{xx}, u(x, 0) = \sin(\pi x)$

The operator A in $A[u] = 0$ is then:

$$A[u] = \begin{cases} f(u) & C = \emptyset \text{ (scalar)} \\ (Du - f, Cu - g) & \text{otherwise} \end{cases}$$

Lemma 4.2 (Operator factorization). *Let X be a normed linear space. Every well-posed linear boundary value problem induces a bounded linear operator factorization $L = D_0 + BC : X \rightarrow Y \times Z$, where:*

- $D_0 : X \rightarrow Y$ is the differential operator restricted to the kernel of C (homogeneous constraints);
- $C : X \rightarrow Z$ is the constraint (trace) operator selecting boundary or initial data;
- $B : Z \rightarrow Y$ is the boundary lifting operator satisfying $CB = I_Z$ and $D_0B = 0$.

The operator L is invertible iff the homogeneous problem $D_0u = 0, Cu = 0$ has only the trivial solution $u = 0$.

Proof. For any $u \in X$, write $u = u_0 + Bg$ where $Cu_0 = 0$ and $g = Cu$. Then $Lu = (Du, Cu) = (D_0u_0 + DBg, g)$. Since DB maps $Z \rightarrow Y$, define the Schur complement $S = D_0|_{\ker C}$ (invertible by well-posedness). The inverse is $L^{-1}(f, g) = S^{-1}(f - DBg) + Bg$, which is the Green's representation of Theorem 4.3. $\square$

This factorization makes precise the intuition that the constraint operator C and the differential operator D are not independent components but factors of a single bounded linear operator on X . The (D, C) notation is not merely a pair of operators; it denotes the product structure $D_0 + BC$ in the category of normed linear spaces.

4.2. Green's Representation.

Theorem 4.3 (Green's representation). *For a linear operator $L = (D, C)$, the inverse L^{-1} decomposes as:*

$$L^{-1}[(f, g)](x) = H[g](x) + \int_{\Omega} K(x, \xi) \cdot f(\xi) d\xi$$

where H propagates constraints ($Du = 0, Cu = g$) and K is the Green's kernel ($Du = \delta, Cu = 0$).

Proof. By linearity of L . The homogeneous solution $H[g]$ satisfies $L[H[g]] = (0, g)$. The particular solution with Green's kernel satisfies $L[\int K f] = (f, 0)$. By superposition, $L[H[g] + \int K f] = (f, g)$. $\square$

The kernel type is determined by the order and domain of D :

- $D = d/dx$ on $[a, b]$: Volterra kernel (step function, causal). The integral equation $u(x) = u(a) + \int_a^x f(t) dt$ propagates information forward—the value at x depends only on $[a, x]$.

- $D = -d^2/dx^2$ on $[a, b]$: Fredholm kernel (piecewise linear, symmetric). The Green's function $G(x, \xi) = \min(x, \xi) \cdot (1 - \max(x, \xi))$ couples every point to every other point—the value at x depends on the entire domain.

This distinction is not encoded in the solver. It is encoded in the arity of **Integ**: two arguments dispatch to Volterra, three to Fredholm. The same term constructor, **Integ**, handles both. Adding a fourth argument would dispatch to a boundary-element kernel; adding a noise term would dispatch to a stochastic integral. The constructor generalizes without modifying the solver.

4.3. Integ and Diff as First-Class Terms. A single **Integ** constructor dispatches on the presence of the third argument:

Definition 4.4 (Integ dispatch).

$$\text{Integ}(f, a) \rightarrow \text{Volterra: } F(x) = \int_a^x f(t) \, dt$$

$$\text{Integ}(f, a, b) \rightarrow \text{Fredholm: } F(x) = \int_a^b G(x, t) \cdot f(t) \, dt$$

The evaluator dispatches on the arity of **Integ**. No branching on “what kind of equation is this” occurs: the structure of A determines the evaluation strategy.

Similarly, $\text{Diff}(\text{Lam}(x, b))$ is a term of the calculus, not a primitive escape to a numerical procedure. When applied to a point p , it evaluates the symmetric difference quotient.

4.4. Heat Equation as Repeated BVP.

Theorem 4.5 (Implicit Euler as BVP). *The heat equation $u_t = u_{xx}$ with Dirichlet boundary conditions, discretized via implicit Euler with time step Δt , is equivalent to solving a BVP at each time step:*

$$(I - \Delta t \cdot d^2/dx^2)u^{n+1} = u^n.$$

Proof. The implicit Euler discretization of $u_t = u_{xx}$ is:

$$\frac{u^{n+1} - u^n}{\Delta t} = u_{xx}^{n+1} \implies u^{n+1} - \Delta t \cdot u_{xx}^{n+1} = u^n.$$

Let $D = -d^2/dx^2$ with domain $\{u : u(0) = u(1) = 0\}$ and Green's function $K = D^{-1}$ (the Fredholm kernel). Then:

$$u^{n+1} + \frac{1}{\Delta t} \cdot K[u^{n+1}] = \frac{1}{\Delta t} \cdot K[u^n].$$

Setting $\alpha = 1/\Delta t$ and $A[u] \equiv u + \alpha \cdot K[u] - \alpha \cdot K[u^n]$, the equation $A[u^{n+1}] = 0$ is solved by $\text{Fix}(T(A, \eta))$ with an appropriate η . Each time step is one invocation of the BVP solver. The PDE adds nothing fundamentally new—it applies the BVP operator N_t times. $\square$

5. AUTOMATIC DIFFERENTIATION AS A TERM OPERATOR

5.1. The Wengert Tape. A **Wengert tape** [Wen64] is a sequence of scalar operations recorded during forward evaluation. The backward pass processes the sequence in reverse, propagating partial derivatives via the chain rule.

Within the calculus, the tape is a mutable data structure attached to the evaluation context. Each scalar operation that participates in the computation records an entry:

Definition 5.1 (Tape entry).

$$\begin{aligned} \text{TapeEntry} &::= (op, inputs, value, metadata) \\ \text{where } op &\in \{\text{var}, \text{add}, \text{sub}, \text{mul}, \text{div}, \text{pow}, \text{neg}, \\ &\quad \text{sin}, \text{cos}, \text{exp}, \text{log}, \text{sqrt}, \text{tanh}, \text{relu}\} \\ inputs &\subseteq \mathbb{N} \quad (\text{tape indices of operands}) \\ value &\in \mathbb{R} \quad (\text{cached forward value}) \\ metadata &\quad (\text{operand values for gradient rules}) \end{aligned}$$

Theorem 5.2 (Reverse-mode correctness). *For any expression e built from the tape-tracked operations, the reverse pass computes the exact gradient ∇e with relative error bounded by machine precision.*

Proof. Each operation op has an associated backward rule that is the exact partial derivative. For example:

$$\begin{aligned} \text{add}(a, b) : \quad & da += d_{out}, \quad db += d_{out} \\ \text{mul}(a, b) : \quad & da += d_{out} \cdot b, \quad db += d_{out} \cdot a \\ \text{tanh}(a) : \quad & da += d_{out} \cdot (1 - \tanh^2(a)) \end{aligned}$$

By induction on the structure of the tape (which forms a DAG for feedforward expressions), the reverse accumulation computes $\partial e / \partial x_i$ for all inputs x_i simultaneously. The error is due solely to floating-point arithmetic. The full set of backward rules is given in Appendix A. $\square$

Corollary 5.3 (Complexity). *For an expression with p scalar inputs and n operations, the tape computes all p partial derivatives in $O(n)$ time—independent of p .*

5.2. Grad as a Term.

Definition 5.4 (Grad operator). $\text{Grad} : \mathcal{T} \rightarrow \mathcal{T}$ is the term constructor:

$$\text{Grad}(\text{Lam}(\theta, body))$$

which, when applied to a parameter vector θ_{val} , returns $\nabla_{\theta} body(\theta_{val})$.

The semantics of **Grad** consists of three steps:

- (1) **Create** a fresh tape.
- (2) **Evaluate** $body$ with θ bound to tape-tracked variables, recording all scalar operations on the tape.
- (3) **Execute** the reverse pass, accumulating the gradient.

Theorem 5.5 (Grad is a term operator). *Grad is type-preserving: if $body : X \rightarrow \mathbb{R}$, then $\text{Grad}(\text{Lam}(\theta, body)) : X \rightarrow X^*$. The gradient is computed entirely within the term algebra without external procedure calls.*

Proof. The forward pass records operations on the tape. The backward pass reads the tape in reverse order. Both are implemented as reduction rules operating on terms. No external procedure—no call to an AD library—is invoked. The entire computation lives within the reduction system of Section 2. $\square$

The operational realization of **Grad** is a two-component system. A **Tape** structure maintains a Wengert list of **TapeEntry** objects. A **TapeVar** wraps a scalar value with a tape reference and overloads the arithmetic operators ($+$, $-$, $*$, $/$, exponentiation) and elementary functions ($\sin$, $\cos$, $\exp$, $\tanh$, relu). Each operation calls

`record(op, result_val, left_id, right_id, ...)`

appending an entry to the tape. The forward pass builds the tape; the backward pass replays it in reverse, seeding the last entry with gradient 1.0 and propagating via the chain rule (Appendix A).

The key design choice is that **TapeVar** operations are syntactically identical to ordinary arithmetic. An expression such as $\theta_1 \cdot x + \theta_2$ evaluates to a scalar when **TapeVar** carries no tape reference (prediction mode), and builds the full computation graph when it carries an active tape (training mode). This zero-overhead switching enables a single forward function to serve both prediction and gradient computation, without duplicating the model definition.

6. DEEP EQUILIBRIUM MODELS AS NESTED FIXED POINTS

6.1. Forward Pass as Fix. A Deep Equilibrium Model [BKK19] defines a layer implicitly:

$$z^* = f(z^*, x, \theta).$$

In the calculus, this is:

$$z^* = \text{Fix}(\text{Lam}(z, f(z, x, \theta))).$$

The evaluation rule $\text{Fix}(f) \rightarrow f \text{ Fix}(f)$ generates the sequence $z_{k+1} = f(z_k, x, \theta)$. Convergence to a fixed point z^* depends on the contractivity of f with respect to z .

6.2. Gradient as Fix.

Theorem 6.1 (DEQ gradient via implicit function theorem). *The gradient of a loss $\mathcal{L}(z^*)$ with respect to parameters θ , where $z^* = \text{Fix}(\text{Lam}(z, f(z, x, \theta)))$, is:*

$$\frac{\partial \mathcal{L}}{\partial \theta} = g^* \cdot \frac{\partial f}{\partial \theta}$$

where the adjoint g^* satisfies:

$$g^* = \text{Fix}\left(\text{Lam}\left(g, \frac{\partial \mathcal{L}}{\partial z^*} + g \cdot \frac{\partial f}{\partial z}\right)\right).$$

Proof. From $z^* = f(z^*, x, \theta)$, differentiate with respect to θ :

$$\begin{aligned} \frac{\partial z^*}{\partial \theta} &= \frac{\partial f}{\partial z} \cdot \frac{\partial z^*}{\partial \theta} + \frac{\partial f}{\partial \theta} \\ \Rightarrow (I - \frac{\partial f}{\partial z}) \cdot \frac{\partial z^*}{\partial \theta} &= \frac{\partial f}{\partial \theta} \\ \Rightarrow \frac{\partial z^*}{\partial \theta} &= (I - \frac{\partial f}{\partial z})^{-1} \cdot \frac{\partial f}{\partial \theta}. \end{aligned}$$

Then:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \theta} &= \frac{\partial \mathcal{L}}{\partial z^*} \cdot \frac{\partial z^*}{\partial \theta} \\ &= \frac{\partial \mathcal{L}}{\partial z^*} \cdot (I - \frac{\partial f}{\partial z})^{-1} \cdot \frac{\partial f}{\partial \theta} \\ &= g^* \cdot \frac{\partial f}{\partial \theta}, \end{aligned}$$

where $g^* = \frac{\partial \mathcal{L}}{\partial z^*} \cdot (I - \frac{\partial f}{\partial z})^{-1}$.

Solving for g^* :

$$\begin{aligned} g^* \cdot (I - \frac{\partial f}{\partial z}) &= \frac{\partial \mathcal{L}}{\partial z^*} \\ \Rightarrow g^* &= \frac{\partial \mathcal{L}}{\partial z^*} + g^* \cdot \frac{\partial f}{\partial z}. \end{aligned}$$

This is a fixed-point equation in g^* . Hence $g^* = \text{Fix}(\text{Lam}(g, \partial \mathcal{L} / \partial z^* + g \cdot \partial f / \partial z))$. $\square$

Corollary 6.2 (Forward-backward unification). *For DEQ models, both the forward pass and the gradient computation are instances of $\text{Fix}(T)$. The chain rule is replaced by another fixed point.*

Theorem 6.3 (Memory complexity). *The DEQ gradient computation stores only the final fixed point z^* , not the intermediate iterates $z_1, \dots, z_{K-1}$. The memory complexity is $O(1)$ in the number of forward iterations.*

Proof. The adjoint equation depends on f only through $\partial f / \partial z$ evaluated at z^* . The intermediate values z_k are not needed. The Fix expansion for the gradient produces its own iterates g_k , but these do not require storing $z_1, \dots, z_{K-1}$. $\square$

The nested Fix structure reveals a fundamental equivalence: training a DEQ model consists of two nested invocations of the T-transform iteration. The outer loop trains parameters ($\theta \leftarrow \theta - \eta \cdot \nabla_{\theta} \mathcal{L}$), and the inner loop computes the adjoint ($g_{k+1} = \partial \mathcal{L} / \partial z^* + g_k \cdot \partial f / \partial z$). Both are $\text{Fix}(T)$ for different choices of A . The original DEQ formulation [BKK19] requires solving a linear system $(I - \partial f / \partial z)^{-1}$ or using an iterative solver; in our calculus, the linear solve is the Fix expansion—no distinction between “direct” and “iterative” solvers exists. The solver is the primitive.

7. IMPLEMENTATION

The entire formal system is implemented in approximately 1,500 lines of code with a single numerical dependency. The T-transform and its fixed-point solver together require 29 lines.

7.1. From Theory to Computation. The computational architecture mirrors the theoretical structure: term constructors (Section 2.1) become datatype definitions, reduction rules (Section 2.2) become an evaluator, and the T-transform (Section 3) compiles to a kernel of under 30 lines.

7.2. Differentiation Strategies. The system provides two gradient-computation strategies, both conforming to the same abstract interface:

Definition 7.1 (Model interface). A **model** is a triple $(\theta_0, f, \nabla f)$ where $\theta_0 \in \mathbb{R}^p$ is the initial parameter vector, $f : \mathbb{R}^p \times \mathbb{R} \rightarrow \mathbb{R}$ is the forward function, and $\nabla f : \mathbb{R}^p \rightarrow \mathbb{R}^p$ computes the gradient of a loss functional with respect to θ .

Finite-difference gradient. For any forward function $f(\theta, x)$, the gradient is approximated by central differences:

$$(\nabla_{\theta} \mathcal{L})_i \approx \frac{\mathcal{L}(\theta + h e_i) - \mathcal{L}(\theta - h e_i)}{2h},$$

where e_i is the i -th standard basis vector and $h \approx 10^{-5}$. Complexity is $O(p)$ per gradient evaluation with $O(h^2)$ accuracy. No tracing, no tape, no special syntax is required—any computable function $(\theta, x) \mapsto y$ can be differentiated.

Tape-based gradient. For a forward function expressed in **TapeVar** arithmetic (Section 5), all scalar operations are recorded on a Wengert tape during one forward pass. A single reverse sweep—**backward()**—computes all p partial derivatives simultaneously via the chain rule. Complexity is $O(n)$ where n is the number of scalar operations in the forward pass, independent of p . For a typical neural network layer with weight matrix $W \in \mathbb{R}^{m \times n}$ and bias $b \in \mathbb{R}^n$, the forward pass performs $\approx 2mn + n$ scalar operations, and the reverse pass costs roughly the same—the “cheap gradient principle” [GW08].

Both strategies are instances of the same **Grad** operator (Definition 5.2), differing only in how they evaluate the derivative. The training algorithm is agnostic to this choice.

7.3. Training as T-Transform Iteration. Training a model reduces to the T-transform iteration with $A = \nabla_{\theta} \mathcal{L}$:

```

for  $t = 0, 1, \dots, N - 1$ :
   $g_t \leftarrow \nabla_{\theta} \mathcal{L}(\theta_t)$ 
   $\theta_{t+1} \leftarrow \theta_t - \eta \cdot g_t$ 

```

The parameter η is the learning rate, time step, and damping factor simultaneously (Theorem 3.9). Each iteration is one application of $T_{\nabla \mathcal{L}, \eta}$. The algorithm does not inspect whether g_t came from finite differences or a reverse-mode tape—the gradient is a vector in $\mathbb{R}^p$, and the update is vector subtraction. The model provides g_t ; the trainer provides η and the loop; the T-transform provides the guarantee that $\text{Fix}(T_{\nabla \mathcal{L}, \eta})$ is a stationary point of $\mathcal{L}$.

7.4. PDE Solving.

Equation	A operator	Iterations	Error
$x^3 - 2x - 5 = 0$	Lam ($x, x^3 - 2x - 5$)	9	$f(x^*) = 2.1 \times 10^{-9}$
$y' = y, y(0) = 1$	Picard residual	11	L^2 error 2.0×10^{-5}
$-u'' = 1, u(0) = u(1) = 0$	BVP residual	24	L^2 error 5.4×10^{-9}
Heat eq. (3 steps)	Implicit Euler residual	5/step	$O(\Delta t)$ error

7.5. Neural Network Training.

Task	Model	Parameters	Gradient method
Linear regression ($y = 2x + 1$)	FunctionModel	2	Finite diff.
$\sin(x)$ regression	MLP[1,16,1]	49	Tape ($O(1)$)
Binary classification	Linear + BCE	2	Finite diff.
DEQ tanh fit	DEQ function	3	Finite diff.
Gradient accuracy check	TapeModel	49	Tape vs. FD
CIFAR-10	MLP[256,64,10]	17,162	Tape ($O(1)$)

Result	Value
Linear regression: $\hat{a}, \hat{b}$	1.94, 1.08
$\sin(x)$ regression: final MSE	0.061
Binary classification: accuracy	93.3%
DEQ tanh fit: final MSE	0.009
Tape vs. finite difference: relative error	$\sim 10^{-11}$

The tape-based reverse-mode AD computes exact gradients (relative error $\sim 10^{-11}$ versus finite differences) in $O(1)$ time in the number of parameters, independent of the forward expression size.

7.6. Reproducibility. The complete implementation, including all experiments, is available as ancillary material accompanying this submission. All numerical results reported in this section are reproducible from the provided code.

8. DISCUSSION

8.1. Related Formal Systems. The presentation of Fix as a primitive constructor differs from its standard treatment in domain theory [Sco76, Plo77], where the least fixed-point operator $\mathbf{Y}$ is definable or axiomatized as $\mathbf{Y}(f) = f(\mathbf{Y}(f))$ in a typed setting. Our approach is closer to the spirit of the untyped λ -calculus, where recursion is freely available, but we retain the explicit Fix constructor to make the fixed-point structure visible to the syntax.

The (D, C) decomposition echoes the formal theory of boundary value problems in functional analysis [Sta98], but we internalize the decomposition within the term calculus as an operator factorization (Lemma 4.2): D and C are not merely a pair of operators but factors $D_0 + BC$ of a single bounded linear operator on a normed space.

The present paper provides an operational semantics (reduction rules) and a denotational semantics restricted to complete metric spaces (Banach fixed-point interpretation, Remark 2.3). A full denotational semantics in the sense of domain theory or traced monoidal categories—with soundness and completeness relative to the operational semantics—remains for future work (Section 8.2).

8.2. Future Work.

- (1) **Denotational semantics.** The present paper provides an operational semantics (reduction rules) and a Banach fixed-point interpretation of `Fix` (Remark 2.3). A full denotational semantics—with soundness and completeness theorems relating the reduction system to a domain-theoretic or traced monoidal category model—is the principal next step. The T-transform $T[u] = u - \eta \cdot A[u]$ is a natural transformation in the category of normed vector spaces, and `Fix` is a trace in the sense of Joyal, Street, and Verity [JSV96], suggesting that traced monoidal categories and the geometry of interaction provide the right semantic foundation.
- (2) **Type theory.** The current system is untyped. Adding a type system with `Fix` as a type constructor (μ -types) would connect to domain theory and inductive/coinductive types [Pie02, Har16].
- (3) **Convergence theory.** The formal correctness of $\text{Fix}(T)$ (Theorem 3.2) is independent of convergence. A complete convergence theory would characterize the class of operators A for which $T_{A,\eta}$ is a contraction, and the optimal η schedules for non-convex operators.
- (4) **Compiler.** The term evaluator is an interpreter. A compiler from Term expressions to optimized numerical code (JIT or ahead-of-time) would eliminate the interpreter overhead for deployed models.
- (5) **Probabilistic extension.** Adding a noise term $T[u] = u - \eta \cdot A[u] + \sqrt{2\eta} \cdot \xi$ yields Langevin dynamics. $\text{Fix}(T)$ becomes the stationary distribution—Bayesian inference as fixed point.
- (6) **Higher-order operators.** The current calculus applies `Integ` and `Diff` to first-order terms. Extending to higher-order operators (biharmonic, polyharmonic) follows the same template: each operator is a Term whose application dispatches to the corresponding Green’s kernel.

8.3. Comparison with Machine Learning. While the contributions of this paper are formal, they bear directly on computational practice. The standard neural network training loop—forward pass, loss, backward pass, parameter update—is exactly the T-transform iteration $u_{t+1} = u_t - \eta \cdot A[u_t]$ with $A = \nabla_{\theta} \mathcal{L}$. That this is not a coincidence but a formal identity means that any improvement to fixed-point convergence (better η schedules, preconditioners, acceleration methods like Anderson mixing [And65]) immediately improves training across all model architectures, without modifying the model.

9. CONCLUSION

We have defined a term calculus in which the fixed-point combinator `Fix` is a primitive syntactic constructor, the T-transform $T[u] = u - \eta \cdot A[u]$ is the canonical affine residual operator embedding operator equations into discrete dynamical systems, and the single construct $\text{Fix}(T)$ solves scalar equations, initial value problems, boundary value problems, PDEs, and the gradient computations of deep equilibrium models—all as instances of the same operator flow.

The central insight is that **the boundary between “solving an equation” and “training a model” is not a mathematical distinction but an artifact of our computational tools.** What is unified is not the structure of the equations but the

evolution in function space that solves them. When Fix is a primitive of the language, solving *is* computing.

Acknowledgments. The author thanks the LMCS editors and anonymous reviewers for their constructive feedback.

APPENDIX A. BACKWARD RULES FOR THE WENGERT TAPE

The following rules are applied during the reverse pass of the tape. Each rule receives the output gradient d_{out} and accumulates into the gradients of the inputs.

Operation	Backward rule
$\text{add}(a, b)$	$da += d_{out}; db += d_{out}$
$\text{sub}(a, b)$	$da += d_{out}; db -= d_{out}$
$\text{mul}(a, b)$	$da += d_{out} \cdot b; db += d_{out} \cdot a$
$\text{div}(a, b)$	$da += d_{out}/b; db -= d_{out} \cdot a/b^2$
$\text{pow}(a, b)$	$da += d_{out} \cdot b \cdot a^{b-1} \ (a \neq 0);$ $db += d_{out} \cdot a^b \cdot \ln a \ (a > 0)$
$\text{neg}(a)$	$da -= d_{out}$
$\text{sin}(a)$	$da += d_{out} \cdot \cos a$
$\text{cos}(a)$	$da -= d_{out} \cdot \sin a$
$\text{exp}(a)$	$da += d_{out} \cdot e^a$
$\text{log}(a)$	$da += d_{out}/a$
$\text{sqrt}(a)$	$da += d_{out}/(2\sqrt{a})$
$\text{tanh}(a)$	$da += d_{out} \cdot (1 - \tanh^2 a)$
$\text{relu}(a)$	$da += d_{out} \text{ if } a > 0, \text{ else } 0$

The backward rules are the exact partial derivatives. The only source of error in the computed gradient is floating-point arithmetic.

REFERENCES

- [And65] Donald G. Anderson. Iterative procedures for nonlinear integral equations. *Journal of the ACM*, 12(4):547–560, 1965. doi:10.1145/321296.321305.
- [Ban22] Stefan Banach. Sur les opérations dans les ensembles abstraits et leur application aux équations intégrales. *Fundamenta Mathematicae*, 3:133–181, 1922. doi:10.4064/fm-3-1-133-181.
- [Bar84] Henk P. Barendregt. *The Lambda Calculus: Its Syntax and Semantics*, volume 103 of *Studies in Logic and the Foundations of Mathematics*. North-Holland, 1984.
- [BKK19] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. Deep equilibrium models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [CRBD18] Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
- [GW08] Andreas Griewank and Andrea Walther. *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*. SIAM, 2nd edition, 2008. doi:10.1137/1.9780898717761.
- [Har16] Robert Harper. *Practical Foundations for Programming Languages*. Cambridge University Press, 2nd edition, 2016. doi:10.1017/CB09781316576892.

- [JSV96] André Joyal, Ross Street, and Dominic Verity. Traced monoidal categories. *Mathematical Proceedings of the Cambridge Philosophical Society*, 119(3):447–468, 1996. doi:10.1017/S0305004100074338.
- [Kle38] Stephen Cole Kleene. On notation for ordinal numbers. *Journal of Symbolic Logic*, 3(4):150–155, 1938.
- [Kle52] Stephen Cole Kleene. *Introduction to Metamathematics*. D. Van Nostrand, 1952.
- [Pie02] Benjamin C. Pierce. *Types and Programming Languages*. MIT Press, 2002.
- [Plo77] Gordon D. Plotkin. Lcf considered as a programming language. *Theoretical Computer Science*, 5(3):223–255, 1977. doi:10.1016/0304-3975(77)90044-5.
- [Sco76] Dana S. Scott. Data types as lattices. *SIAM Journal on Computing*, 5(3):522–587, 1976. doi:10.1137/0205037.
- [Sta98] Ivar Stakgold. *Green’s Functions and Boundary Value Problems*. Wiley, 2nd edition, 1998.
- [Wen64] Robert E. Wengert. A simple automatic derivative evaluation program. *Communications of the ACM*, 7(8):463–464, 1964.